Université de Strasbourg

UFR de mathématique et d'informatique

Génération de codes par IA, promesses et périls

Ugo Berton, Ahmet Berk Ateş, Antoine Gautheron

Encadrant : M. Basile Sauvage

3 mai 2024



Table des matières

1 Introduction **2**

2 Historique et tour d'horizon des outils d'aide au développement jusqu'à l'IA **3**

2.1 Court historique de la programmation 3

2.2 L'auto-complétion et la coloration syntaxique 3

2.3 Les générateurs de structures 4

2.4 La documentation simplifiée 4

2.5 L'outil ultime regroupant tous les autres : L'IDE 4

3 Le fonctionnement de l'IA générative **5**

3.1 Les réseaux de neurones artificiels 5

3.2 Le deep learning et le large language model 7

3.3 Les transformateurs 8

3.4 Le fine tuning 10

4 Apports et influences de l'IA générative **11**

4.1 L'IA, les outils et les développeurs 11

4.2 Le prompting 13

4.3 Les risques et périls 14

5 Conclusion **15**

1 Introduction

Depuis les années 1950 et l'invention des premiers langages de programmation modernes avec le Fortran, les métiers dans le secteur du développement logiciel ont évolué en adéquation avec les évolutions technologiques : création de l'interface graphique (Apple Lisa, Steve Jobs, 1983), invention du low code, l'arrivée des outils Git, ... etc.

Au fil du temps, la conception, la construction, l'amélioration et la maintenance des logiciels sont devenus de plus en plus aisés pour les développeurs. Cependant, au cours du temps et des multiples bouleversements, une constante est restée : le code était réalisé par un humain. Mais une des dernières innovations en date, est actuellement en train de remettre en cause ce fait jusqu'alors établi : l'intelligence artificielle générative. En effet, ses promesses d'un gain de productivité massif font miroiter une révolution à venir. Néanmoins, toute évolution s'accompagne de ses mauvais côtés dont il faut se méfier et l'intelligence artificielle n'y fait pas exception.

En parallèle à ces enjeux, une course se joue actuellement entre les géants mondiaux du secteur (OpenAI, Google, Microsoft, ...), afin de s'imposer comme le futur acteur incontournable de cette technologie qui pourrait bouleverser de nombreux domaines. Dans ce rapport, nous allons uniquement nous concentrer sur celui de la programmation et plus particulièrement sur le thème de la génération de code par ces dites IA.

Ainsi, nous allons nous interroger sur le fonctionnement de l'IA générative et essayer de comprendre comment elle s'est imposée comme un outil du développeur dans le cadre de la génération de code. Par la même occasion, nous nous efforcerons de mettre en lumière ses promesses pour l'avenir et les dangers auxquels elle nous expose. Pour ce faire, nous allons débuter par un rapide historique et tour d'horizon des outils du développeur jusqu'à l'IA générative. Par la suite, nous intéresserons au fonctionnement de cette dernière. Et enfin, nous terminerons avec les gains mais aussi les menaces que cette technologie apporte avec elle.

2 Historique et tour d'horizon des outils d'aide au développement jusqu'à l'IA

2.1 Court historique de la programmation

La programmation informatique et par extension les outils pour faciliter son approche ne sont pas apparus ces dernières années avec l'arrivée de l'IA, ils remontent à bien plus loin dans le temps. En effet, on considère de manière générale que c'est Ada Lovelace, une comtesse anglaise du XIXème siècle, qui a écrit le tout premier programme informatique [1]. Cependant, à cette époque, elle n'avait posé que les balbutiements de la pratique avec comme seuls outils du papier et un crayon.

Ce n'est véritablement qu'un siècle plus tard, dans la seconde moitié du XXème siècle, que les premiers outils vont être créés : il s'agit du moment où les premiers ordinateurs, qui sont des interfaces numériques, sont apparus. Ces outils étaient alors réservés à des professionnels et peu développés. Néanmoins avec le temps et la démocratisation des ordinateurs qui ont répandu la pratique de la programmation, leur nombre a progressivement augmenté et leur panel de fonctions s'est étoffé. Aujourd'hui, nous pouvons citer l'auto-complétion, la coloration syntaxique ou encore la documentation simplifiée comme outils d'aide à la programmation très répandus.

Décortiquons ces outils et pour ce faire, débutons par l'auto-complétion et la coloration syntaxique qui sont de nos jours considérés comme des standards.

2.2 L'auto-complétion et la coloration syntaxique

Pour le premier, nous pouvons en retrouver un exemple en programmation orientée objet (POO) avec l'utilisation de méthodes. Dès lors qu'un point est écrit, signifiant l'emploi d'une méthode sur une instance d'un objet, un menu déroulant s'ouvre automatiquement et affiche celles disponibles pour cedit objet. Cette fonctionnalité qui peut sembler basique est en réalité très pratique car d'un côté elle sert d'aide-mémoire au programmeur mais de l'autre elle empêche également les erreurs de recopie qui peuvent être très compliquées à debugger.

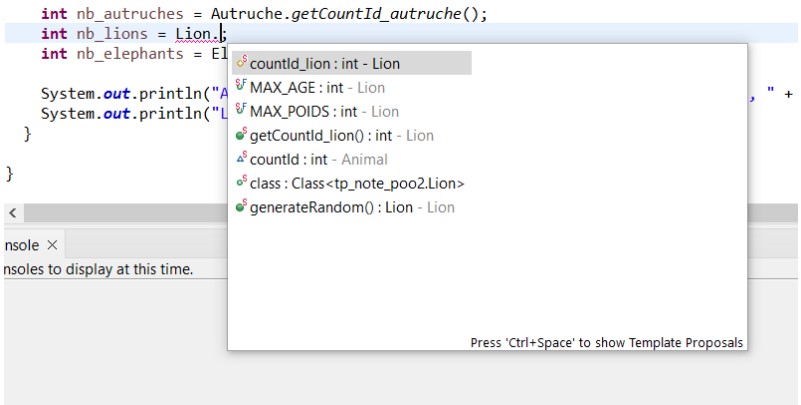


FIGURE 1 – Illustration de l'affichage des méthodes disponibles pour un objet au sein d'Eclipse.

En ce qui concerne le second outil, il s'illustre comme son nom l'indique en venant colorier le code, c'est aussi simple que cela. Toutefois, c'est au même titre très utile : les différentes structures syntaxiques du langage de programmation étant mises en évidence, le code gagne ainsi en visibilité et en clarté pour le programmeur. De cette manière, ce dernier gagne en productivité en réduisant les erreurs de saisie et en améliorant la perception des éléments actifs du programme en cours d'écriture : si une fonction a pour type de retour un entier, symbolisé par un int colorié en jaune, et si cette même fonction retourne un caractère, nous pouvons ainsi rapidement comprendre l'erreur et la corriger. Il existe d'autres outils qui poursuivent ces mêmes objectifs de clarté et d'efficacité. Dans la pratique, ils sont souvent employés en association avec la coloration syntaxique. Nous pouvons mentionner l'indentation automatique, le masquage et démasquage de blocs ou encore la mise en relief d'un délimiteur (parenthèse, crochet, accolade).

2.3 Les générateurs de structures

Continuons avec un outil générant de petites parties de code que l'on pourrait apparenter à un « générateur de structure ». Cet outil, même s'il « crée du code », n'est en rien comparable à l'IA générative. Il a pour seule fonction de fournir des structures types d'un langage cible. En quelques sortes, il nous donne un modèle à développer par la suite. Un exemple peut encore une fois être trouvé en POO, il s'agit de la génération des getter et setter des attributs. En effet, ils ont tous la même forme. Cette fonctionnalité permet ainsi au programmeur de gagner du temps en automatisant des tâches simples et répétitives.

2.4 La documentation simplifiée

Enfin, terminons par la documentation simplifiée. Celle-ci est produite après analyse du code par un logiciel spécifique. Le résultat sera une documentation normalisée que les autres programmeurs pourront consulter afin de rapidement comprendre un code dans son ensemble. De nouveau, cela permet un gain de temps se transformant en gain de productivité pour les programmeurs. Nous pouvons citer Doxygen et son système de balises comme faisant partie de ce type de logiciel [2].

2.5 L'outil ultime regroupant tous les autres : L'IDE

Tous ces outils qui étaient autrefois séparés se sont rassemblés dans les IDE (Environnement de développement intégré) qui sont aujourd'hui la norme dans la programmation [3]. Par exemple, nous pouvons citer Eclipse ou VsCode. Ils permettent d'avoir toutes ces fonctionnalités centralisées au sein d'un même logiciel, en les simplifiant même parfois. Ainsi avec VsCode, en ouvrant un répertoire, il détecte automatiquement la présente d'un dépôt Git, met en place une signalétique pour savoir quels fichiers sont modifiés ou non suivis et permet de réaliser des commit et des push via une interface simplifiée. En même temps, les IDE continuent à évoluer en accueillant les dernières avancées en date comme avec l'IA [4]. Par exemple, nous avons Copilot qui est utilisé par notre enseignant de programmation web comme assistant d'arrière-plan ; il documente automatiquement son code ou lui suggère des morceaux à ajouter ou à modifier lors de ses démonstrations.

Mais comment fonctionne ce nouvel outil, qui à la différence des précédents vient assister le programmeur sur l'aspect sémantique et non syntaxique du code ?

3 Le fonctionnement de l'IA générative

L'IA, selon l'encyclopédie en ligne Larousse [5], représente l'ensemble des théories et des techniques visant à développer des machines capables de simuler l'intelligence humaine. Contrairement à une approche algorithmique classique, elle modélise et s'adapte au problème à résoudre, reproduisant ainsi un comportement normalement associé aux organismes vivants. Le terme "générative" appliqué à l'IA fait référence à sa capacité à produire du contenu, qu'il s'agisse de sons, de textes, d'images ou de vidéos. Cette capacité repose sur la création de contenu à partir d'une partie préexistante ou d'instructions spécifiques fournies à l'IA [6]. Dans notre contexte, nous nous intéressons particulièrement aux agents conversationnels capables de répondre à un texte en produisant un texte en retour (text to text) [7].

Pour appréhender le fonctionnement de l'IA générative, il est nécessaire de comprendre le concept de réseaux neuronaux.

3.1 Les réseaux de neurones artificiels

Ces derniers constituent l'architecture de base de nombreuses applications d'intelligence artificielle

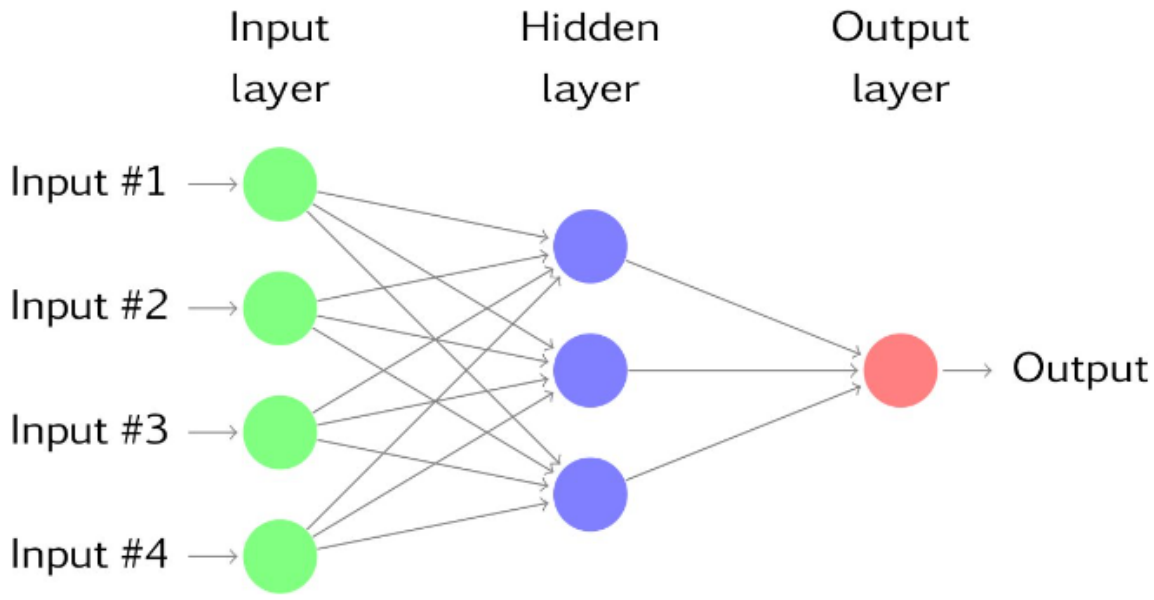


FIGURE 2 – Représentation d'un réseau de neurones [8]

Les réseaux de neurones artificiels sont constitués de couches nodales. Comme observé sur la représentation simplifiée des réseaux neuronaux, au commencement, se trouve une couche d'en-

trée. Elle est suivie par des couches intermédiaires, communément appelées "couches cachées" qui s'étendent entre la couche d'entrée et la couche de sortie. Chacun de ces nœuds sont en fait des vecteurs. Dans le but de simplifier l'explication, nous continuerons à utiliser le terme "nœud" pour décrire ces vecteurs. Ces derniers sont liés avec tous les nœuds de la prochaine couche par des fonctions [9].

Les liaisons entre les nœuds dans les réseaux neuronaux sont établies grâce à des poids, également appelés paramètres. Ces poids sont ajustables et sont soumis à un processus d'entraînement afin d'optimiser les performances du réseau. À chaque entrée d'un neurone, un poids modifiable, noté généralement comme "w", est associé [10]. L'entrée d'un nœud a pour entrée la somme des sorties des neurones qui lui sont connectés, et pour sortie entre chaque couche, une fonction d'activation existe. Il s'agit d'une fonction prenant comme variable un paramètre et permettant aux neurones, s'ils dépassent un certain seuil (marge d'erreur), d'émettre un signal [11].

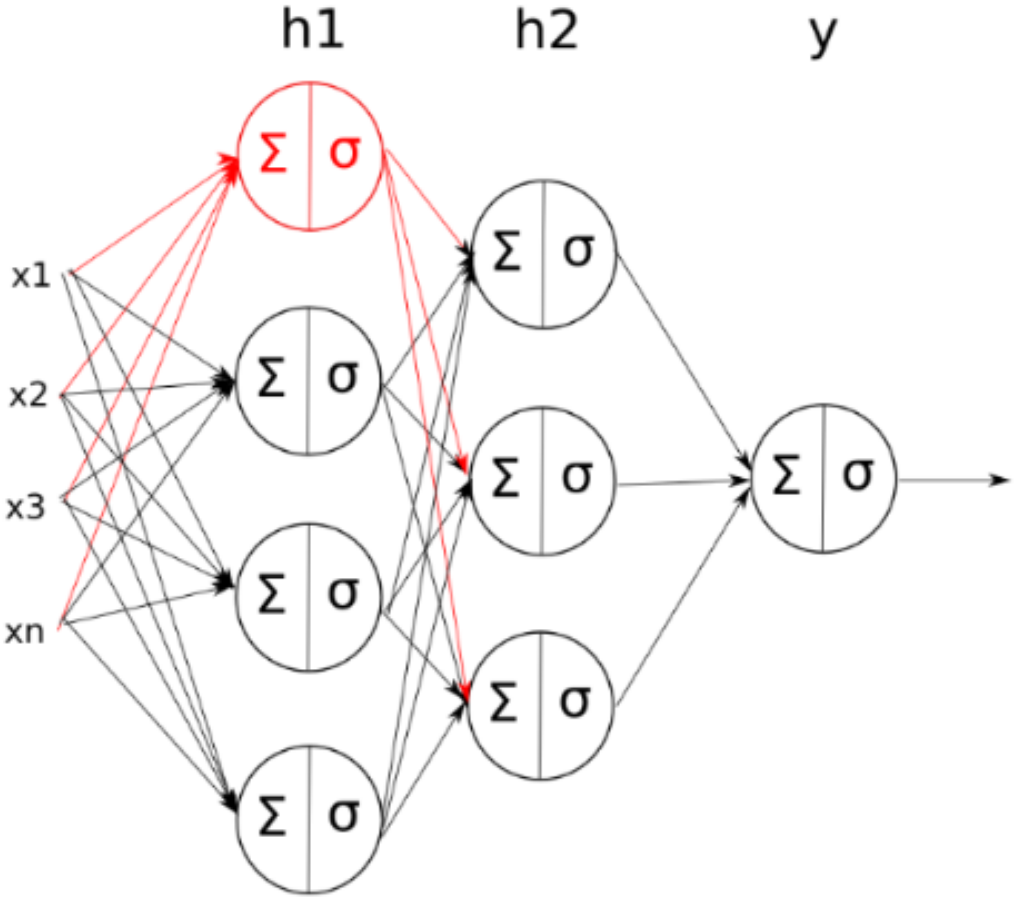


FIGURE 3 – Représentation d'un réseau de neurones avec ses entrées et sorties [10]

Sur cette figure, les nœuds des couches d'entrées ont pour entrée la somme des x_n pondérée par le poids du nœud. Chaque nœud, de la couche d'entrée h1 applique une fonction d'activation sur leur entrée. Ensuite, les nœuds de la couche h2 appliquent une fonction d'activation sur leur

entrée qui est la somme des sorties des nœuds qui lui sont connectés, pour finir le nœud de la couche de sortie applique aussi cette fonction. Cette structure de pondération et d'activation des entrées permet au réseau neuronal de modéliser des relations complexes et de réaliser des tâches de prédiction et de classification. Tout cela en adaptant dynamiquement son comportement en fonction des données d'entrée.

3.2 Le deep learning et le large language model

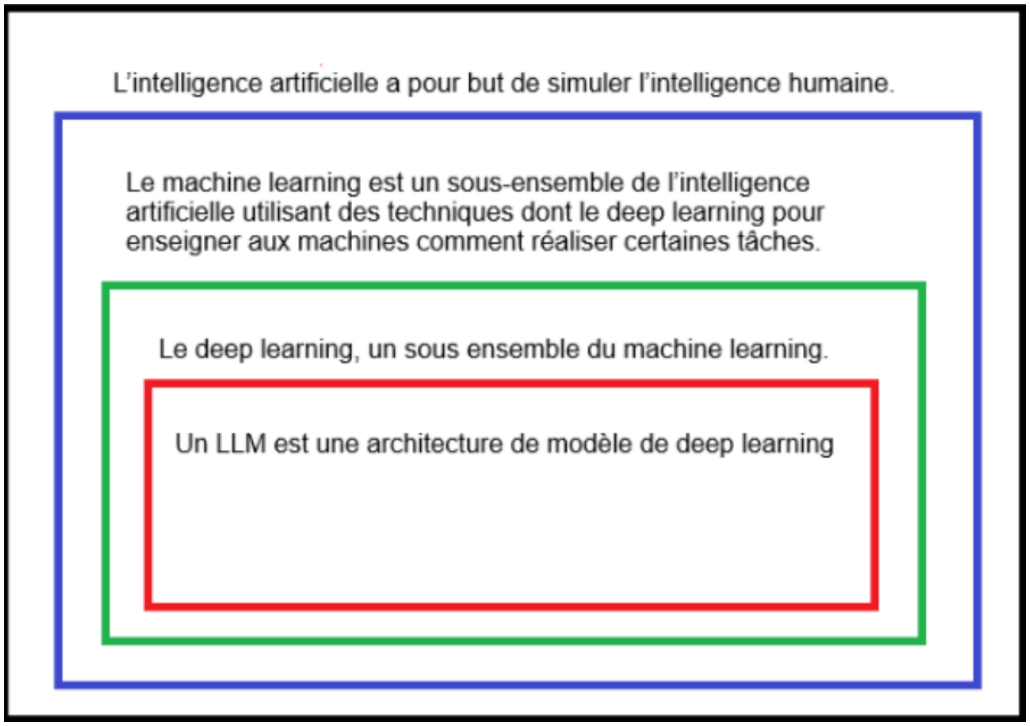


FIGURE 4 – Schéma explicatif des sous-ensembles des concepts de l'IA sorties

Le deep learning, faisant partie du domaine du machine learning, repose sur l'utilisation de réseaux neuronaux artificiels pour réaliser des tâches complexes. Cette approche est qualifiée de "profonde" en raison de la présence de plusieurs couches dans la structure des réseaux neuronaux, notamment des couches masquées, d'entrée, de sortie [12].

Le machine learning, quant à lui, constitue une sous-catégorie de l'intelligence artificielle qui englobe diverses techniques, dont le deep learning. Son principe fondamental est de permettre aux machines d'acquérir des connaissances à partir de l'expérience et des données, afin d'améliorer leur performance dans l'exécution de tâches spécifiques [13].

Les larges languages models (LLM) représentent une application spécifique du deep learning dans le domaine du traitement du langage naturel. En combinant les principes fondamentaux du deep learning, il utilise des réseaux neuronaux artificiels pour des tâches complexes, avec des techniques avancées de traitement du langage naturel.

D'après la conférence de Mirella Lapata qui a été filmée à la Royal Institution le 29 septembre

2023, en collaboration avec l'Institut Alan Turing [14], les LLM sont construits sur la base d'un composant appelé "transformer" en anglais et transformateur en français, qui est une architecture novatrice introduite dans le domaine du traitement du langage naturel.

Cette architecture permet aux LLM de capturer efficacement les dépendances entre les phrases ou les textes à long terme dans les données séquentielles. Les LLM sont donc souvent entraînés sur de vastes ensembles de données textuelles pour prédire le prochain mot dans une séquence de texte donnée. Ces ensembles sont des ouvrages en ligne, des forums comme Overflow, Reddit etc.

En effet, sur les textes qu'il s'entraîne, il effectue une analyse probabiliste sur ces données. Par rapport à la probabilité de ces mots dans le texte, il arrive à les différencier et prédire le prochain mot du texte. Nous avons énoncé qu'il arrive à calculer le mot ou les phrases ou les mots les plus probables pour finir ou compléter un texte mais comment cela se fait-il, de plus, comment le LLM ne perd pas le fil du contexte ?

Pour répondre à ces questions, expliquons l'architecture des transformateurs.

3.3 Les transformateurs

Avant l'arrivée des transformateurs, les réseaux neuronaux qui s'occupaient du traitement du langage naturel utilisaient une architecture d'encodage, décodage simple [15]. C'est-à-dire, la séquence d'entrée est entièrement lue et traitée par les réseaux pour être transformée à une représentation mathématique, des vecteurs notamment pour faire abstraction des données d'entrée. Par la suite l'abstraction de ces données d'entrée est décodée dans l'ordre de l'entrée des données pour arriver à un résultat. De plus, étant donné que les séquences d'entrées sont traitées dans l'ordre et un par un, ce processus est long et nécessite de ne pas oublier le début des entrées pour que le contexte de la phrase reste le même. De cette manière, les anciens modèles prenaient du temps, et il pouvait exister une perte de dépendance entre des données se situant à des bouts de texte lointains.

D'après l'article "Attention is all you need" [16], les couches dans un modèle Transformer peuvent être considérées comme des réseaux neuronaux avec des fonctionnalités en plus, conçues pour traiter des séquences de données et gérer les dépendances à long terme dans les tâches de traitement du langage naturel. Voici une description de ces couches et de leur fonctionnement, figure 5.

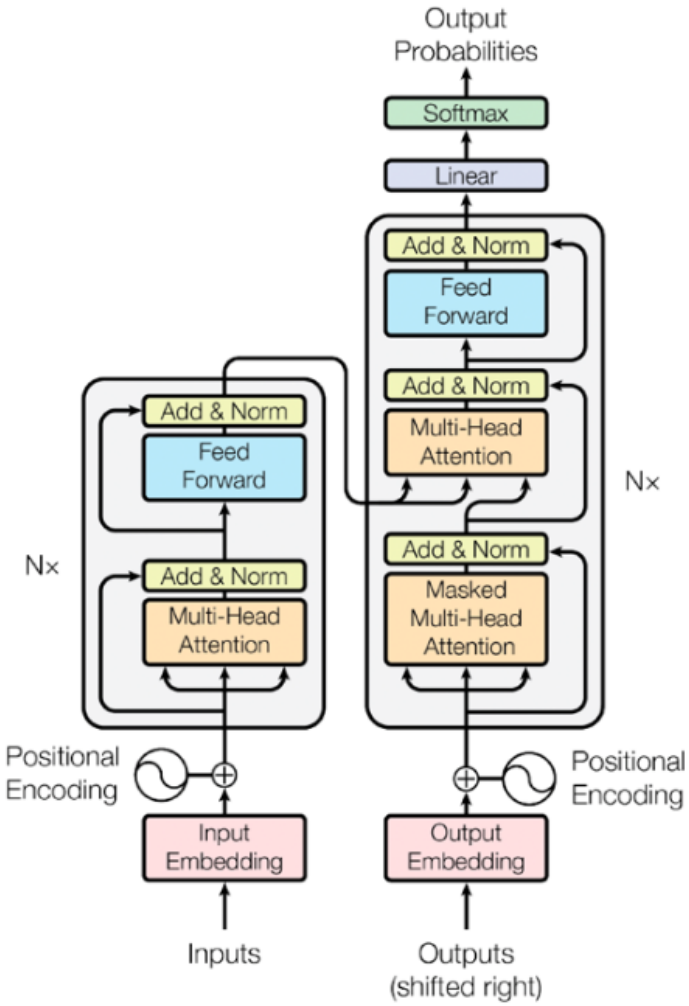


FIGURE 5 – Représentation des couches d'un transformateur [16]

Les étapes clés et les composants qui interviennent entre l'entrée des données et la conclusion du processus dans un modèle de transformateur typique :

1. Couches de Embedding :
   - Input Embedding : chaque token d'entrée est converti en un vecteur via un processus d'embedding, qui transforme des éléments discrets (mots ou caractères) en vecteurs denses de caractéristiques continues.
   - Positional Encoding : après les embeddings, un codage positionnel est ajouté pour intégrer l'information de la séquence, indispensable pour que le modèle sans récurrence puisse comprendre l'ordre des mots dans la séquence.

2. Les bloques de transformateurs :
    — Multi-head Attention : chaque bloc de transformateur comporte une ou plusieurs couches d'attention à multi-têtes. Cette architecture permet au modèle de se concentrer sur différentes parties du texte et de comprendre les relations entre tous les mots d'une phrase, indépendamment de leur position relative. Prenons l'exemple donné par AWS, [17], "Speak no lies", "He lies down", respectivement le premier signifie ne dis pas de mensonge tandis que l'autre il se couche. Le contexte ou "lies" est utilisé est important, le mécanisme d'auto-attention regroupe donc les jetons qui sont nécessaire pour le traitement de "lies".
    — Feed-forward Networks : après le processus d'attention, les informations sont passées à travers un réseau de neurones comme sur figure 2.

3. Output Processing : Une dernière étape de calculs sur les vecteurs existe pour choisir le jeton le plus proche à générer. A la sortie des couches de réseaux de neurones appliquant des transformations linéaires aux données à la sortie des blocs transformateurs. Les résultats récupérés des couches linéaires sont appelés "logits" et sont convertit en une distribution de probabilités qui permet au modèle de choisir le jeton qui est le plus probable.
    — Linear Layers : avant de générer une sortie, les vecteurs traités par les blocs de transformateurs passent par des couches linéaires. Ces couches fonctionnent comme une transformation qui adapte les vecteurs pour préparer la sortie finale.
    — Softmax Layer : elle reçoit les sorties linéaires des couches précédentes, et les soumet à la fonction softmax. Cette fonction transforme les valeurs reçues en une distribution de probabilités, ce qui permet au modèle de déterminer le jeton le plus probable à générer dans le cadre d'une tâche de génération de texte.

Ainsi, il permet le traitement des séquences de données en parallèle tout en considérant l'ensemble du contexte, sans se soucier de l'ordre original de ces derniers. Cette architecture révolutionne le traitement du langage naturel en améliorant la précision, la vitesse et la taille des modèles NLP, permettant de meilleures traductions de langue, meilleures réponses aux demandes de l'utilisateur dont notamment la génération de code.

De manière similaire pour améliorer les performances d'un LLM après son entraînement avec des grandes données, une dernière phase d'entraînement est réalisé et s'appelle le fine tuning.

3.4 Le fine tuning

Le fine-tuning est une méthode consistant au fait de personnaliser les LLM qui ont été entraînés sur des vastes ensembles de données pour des tâches spécifiques en affinant les paramètres du réseau sur un nouveau jeu de données cible [18]. Cela permet au modèle de faire des meilleures prédictions dans un contexte donné.

Le fine tuning est également utilisé pour améliorer le comportement de ces modèles. Notamment, pour que l'agent réponde correctement aux humains, étant donné que les LLM s'entraînent sur des vastes données, il est nécessaire de restreindre les manières dont il peut répondre. Il est restreint que l'agent conversationnel nous réponde vulgairement ou d'une manière déplaisante. Le framework HHH Helpful, Harmless, Honest est appliqué pour répondre à ce besoin. Des réponses à des requêtes de l'IA sont données aux humains et ils choisissent leur réponse préférée [14].

4 Apports et influences de l'IA générative

L'IA générative de code est un outil extraordinaire qui a durablement impacté la société et qui a révolutionné la vie des développeurs. Elle s'est démocratisé avec notamment *Copilot* le 27 octobre 2021 [19] et avec *ChatGPT* le 20 novembre 2022 [20]. Cette avancée étant récente, nous ne pouvons donc pas savoir clairement à quelle échelle s'effectueront les changements qui lui sont liés sur nos vies. Cependant, de nombreuses choses sont déjà apparues avec l'IA générative, que ce soit dans le domaine de l'aide à la programmation des développeurs, des disciplines liées à sont utilisation ou même des problématiques inhérentes lié à ces principes. Nous pouvons alors essayer de les étudier et analyser pour tenter d'avoir une image du paysage informatiques à l'avenir.

4.1 L'IA, les outils et les développeurs

L'IA générative de code est avant tout un outil et une assistance pour les développeurs. Les économistes de Goldman Sachs [21] estiment même que son développement pourrait faire grimper la productivité du travail de l'ordre de 15% au cours des dix prochaines années aux Etats-Unis. L'intelligence artificiel générative de code sert donc à faciliter leur travail en créant des raccourcis avec la création simple, efficace et plutôt optimisée de fonctions annexes ou de *templates* adaptés au besoin de l'utilisateur. Elle possède aussi de nombreuses autres applications, comme la traduction d'un langage à un autre, la génération de tests automatisés et l'apprentissage de nouveaux langages [22].

Cet outil est donc beaucoup plus spécifique et en adéquation avec le développeur que les outils d'aide à la programmations classiques cité dans la première partie (cf. partie 2). Comme le dit Fletcher Previn [23], vice-président senior et DSI de *Cisco*, ce nouvel outil est très intéressant car il n'y avait, avant, aucun moyen de comprimer les délais de développement des logiciels. Pour comparaison [22], prenons donc, dans un premier temps, la correction syntaxique et la détection de bugs potentiels. Dans ce domaine, l'IA va directement analyser l'entièreté du code pour proposer des nouvelles versions en accord avec l'entièreté de la logique du programme. Les corrections seront donc beaucoup plus pertinentes par rapport à juste donner la bonne écriture de tel ou tel instruction ou objet pour les outils classiques. En plus de corrigé, l'IA générative de code peut aussi compléter et même détailler le travail d'un développeur. Contrairement à l'auto-complétion ou à la proposition de méthodes apporté par les anciennes extensions d'IDE, l'IA est en capacité de générer des *templates*, des fonctions, des variables en accord avec la volonté du programmeur ayant renseigné, au moins, une direction de développement avec un nom de classe ou une phrase logique. Nous pouvons notamment illustrer ces propos avec l'extension *Copilot* de *Github* [24] qui va automatiquement proposer à l'utilisateur un bout de code en rapport avec le nom de la variable déjà écrite. Dans l'exemple ci-dessous, toute la partie surlignée en bleue est la création de *Copilot*.

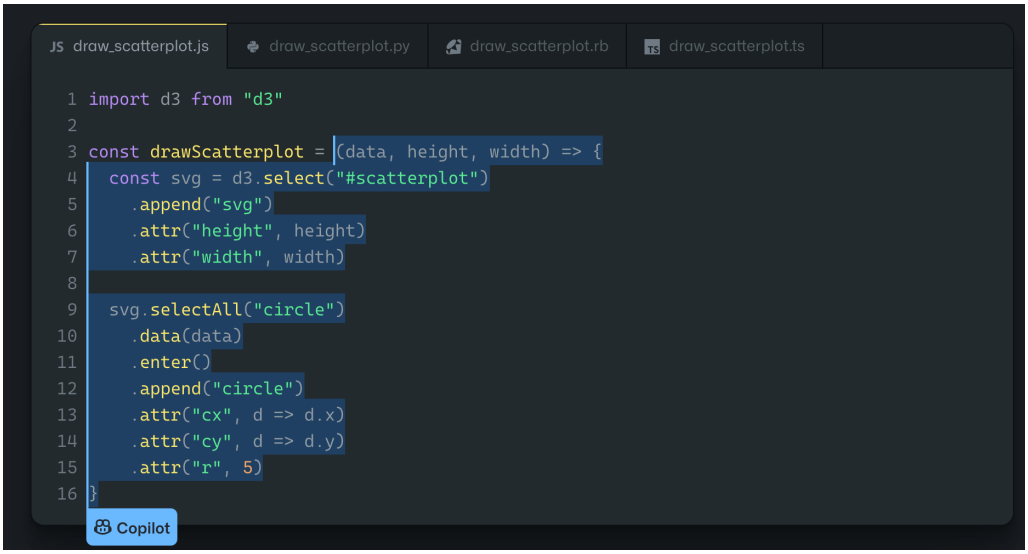


FIGURE 6 – Exemple de code proposé par Copilot [25]

Tandis que un outil classique va seulement proposer, par exemple, la méthode "*select*" de l'objet "*d3*" de cette illustration (cf. ligne 4 - Figure 6).

Les seuls point où l'IA ne sera pas plus efficaces sont dans les domaines qui touchent directement le développeur tel que la coloration syntaxique ou même les cerveaux des programmeurs, qui sont aussi des outils en soi. Seul le développeur saura ce qu'il veut et ce qu'il ne veut pas. Seul lui pourra être touché par la coloration pour mieux se retrouver parmi ces lignes de codes.

L'IA générative de code dépasse donc largement les outils d'aide à la programmation sans intelligence artificielle autant dans la correction et la pertinence du résultat, que dans sa profondeur et son développement. Cependant, le développeur reste seul maître à bord car la fonctionnalité du programme entier, dans le cadre de son projet, ne dépend que de lui. Seul son savoir et ses compétences vont l'amener à privilégier une approche plutôt qu'une autre, ce qu'une IA ne peut prédire sans que celui-ci ait une idée claire de ce qu'il veut. D'où le besoin des anciens outils pour une meilleure compréhension et efficacité du développeur. Ces assistants vont donc proposer une approche plus **syntaxique** alors que l'IA va surtout proposer une analyse plus **sémantique**. Ces deux approches sont donc complémentaires, bien que l'arrivée de l'IA génératrive de code ait révolutionné certains de ces outils.

Nous pouvons par contre noter que les entraînements de ces intelligences (cf. partie 3) se sont faites à partir de forum, de programmes en tout genre et surtout avec l'étude de l'utilisation de ces anciens outils en ce qui concerne *GitHub Copilot* particulièrement [26]. Sans complètement les exclure, car ceux-ci seront toujours utiles, l'IA générative de code est ainsi le digne successeur de ces assistants d'une autre époque.

Cependant, la fiabilité, tant dans la sécurité que dans l'efficacité de ces entraînements reste une question épineuse (cf. partie 4.3).

4.2 Le prompting

Le *prompting*, ou *prompt engineering*, est une discipline développée avec l'arrivée des LLM qui consiste à donner des instructions en langage naturel à l'algorithme de l'IA, très précises et efficaces, afin d'obtenir un résultat directement en accord avec les volontés de l'utilisateur [27]. Plus concrètement, il faut comprendre la façon dont l'algorithme risque de réagir pour anticiper l'entièreté des détails nécessaires à la génération du résultat attendu. Il faut aussi, dans le cadre du code, être plus expérimenté que l'IA sur le thème abordé. C'est ce qui diffère donc d'une requête classique qui va, soit rapporté des détails inutiles, soit être trop brève ou imprécise par rapport au fond de la demande.

Cependant, en quoi un prompt à une IA est plus efficace qu'une requête à un moteur de recherche ? Tout d'abord, l'IA générative va produire une réponse personnalisée à la demande de l'utilisateur, prenant alors en compte le contexte et les "conversations" précédentes [28]. Contrairement à un moteur de recherche qui va montrer les ressources les plus pertinentes à la requête seul, un LLM va aussi prendre en compte dans son algorithme les informations qu'il a en mémoire. Un prompt se devra alors de tenir compte de cela pour affirmer ou infirmer les informations précédentes. Plus évidente, la plus grosse des différence entre la requête et le prompt est que, cette première va servir uniquement de répertoire à mots clé pour la recherche dans données indexés, alors que cette seconde va surtout servir de base à la construction de contenu en rapport avec celle-ci, toujours en prenant compte de sa mémoire. Ce qui explique en même temps le fait qu'un même prompt ne produira pas exactement le même résultat plusieurs fois d'affilée.

Dans le cadre de notre recherche sur les IA générative de code, cette prise en compte du contexte est primordiale afin de répondre aux requêtes de corrections ou bien d'améliorations. C'est ce qui fait principalement que le résultat sera aussi pertinent, sachant qu'en informatique, il y a de nombreux cas très spécifiques qui ne trouvent pas forcément son pareil. Nous pouvons l'observer avec notamment les forums comme *StackOverflow* qui vont pouvoir aider les utilisateurs, soit en utilisant une réponse préexistante, soit en demandant directement à la communauté. Les défauts de ces méthodes [17] sont donc, comparé à l'IA, la non-personnalisation de la réponse pour la première car cela ne s'appliquera pas forcément au cas présent, le temps de réponse de réponse pour la deuxième car contrairement à un LLM, la réponse n'est pas instantané. De plus, la justesse de la réponse peut être aussi questionnée bien que, comme nous allons voir (cf. partie 4.3), celle des IA générative de code aussi.

Enfin, nous pouvons aussi nous demander s'il n'y aurait pas de meilleure alternative au prompt encore plus rigoureuse pour la génération de code. Des chercheurs [29] ont donc imaginé utiliser une des méthodes les plus pertinentes dans ce cadre là, les spécifications fonctionnelles. En effet, la précision et les détails proposés par ce format devrait être particulièrement adapté à l'IA générative de code, de part la complète mise en contexte, la précision de construction des méthodes et les nombreuses conditions de fonctionnement décrites dedans. Cependant, très peu d'avancées public ont été réalisées dans ce sens bien que certaines ressources [29] avance son potentielle et son grand intérêt. Aujourd'hui, les entreprises utilisent surtout l'alliances des deux à des fins de *smar-testing* [30]. C'est-à-dire que l'IA générative de code va créer des tests à partir de la spécification fonctionnels afin de détecter d'éventuels bugs ou problèmes de conceptions.

4.3 Les risques et périls

Les algorithmes d'IA générative de code sont souvent acclamés pour leur performances novatrice et leur divers milieu d'application, permettant alors une nette optimisation de la productivité. Cependant, de nombreuses choses sont à mentionner quant aux dangers actuels et futurs liés à leur utilisation.

Les codes créés par l'IA sont sujets à de nombreux biais [31]. S'entraînant sur des bases de données et des forums tels que *GitHub* pour *GitHub Copilot* [26], les LLM vont avoir tendance à se baser sur les résultats les plus populaires pour la génération de la réponse, ou bien juste sur le code d'un développeur quelconque. Cependant, aussi populaire soient ces bouts de programmes, ils peuvent être sujet à bugs, erreurs et même désuétude dû à leur ancienneté. L'IA va donc perpétré le code sans pour autant avoir une correction dessus. La sécurité est aussi un point essentiel lié à cela. De nombreuses études et recherches démontrent que les codes générés par IA sont peu ou pas fiables au niveau de la sécurité. « L'IA vient ajouter des failles qu'un humain ne laisserait pas passer en temps normal » [32]. « De plus, le code généré par les IA peut être difficile à auditer et à comprendre, ce qui rend difficile la détection de failles de sécurité. Faute de quoi, seules des IA spécialisées en cybersécurité seront capables de l'analyser » [22]. C'est un énorme problème qui est donc difficilement solvable en l'état actuel des choses. Enfin, bien que l'IA générative de code soit assez sophistiquée et pratique pour des cas simples, l'analyse et la création de code et d'algorithmes complexes restent difficiles et très peu fiables à l'heure actuelle.

Avec ces problèmes techniques, s'ajoutent des problèmes pratiques. En effet, les programmeurs risquent des dépendances excessives aux IA génératives de code. Dû faîte de leur efficacité et leur autonomie, elles pourraient prendre un rôle beaucoup trop important dans le cycle de développement, ce qui entraînerait une paresse est une baisse des compétences chez les développeurs [23] [33]. Il y aurait alors une perte de contrôle sur le contenu de nos applications, ce qui laisserait place à tous les problèmes techniques cités précédemment.

Enfin, l'IA générative de code risque d'avoir un énorme impact sur le métier de programmeur [23], qui devra donc évoluer plus en tant que développeur au sens large du terme. Un programmeur devra donc plutôt acquérir des compétences de gestion, de vision et de conception de projets afin de s'adapter à cette révolution. Quand les tâches simples seront gérer et automatiser, le développeur devra plus avoir un regard de correction et d'expertise afin de mieux guider les LLM dans leur tâches. Plus globalement, l'IA générative de code va donc faire évoluer les métiers liés de près ou de loin à l'informatique dans les prochaines années. « La *GenAI* influencera 4,5 fois plus d'emplois qu'elle n'en remplacera » [23].

L'IA générative de code n'est donc pas exempte de défauts. Actuellement, de nombreux problèmes subsistent et de nouveau seront sûrement découverts à l'avenir. Pour autant, c'est un formidable outil que nous devons manier avec précaution. Par exemple, les chercheurs considèrent tout de même que l'IA peut être utile pour coder des éléments qui ne présentent pas de risque [32]. De plus, avec une prévention et des pré-requis (cf. partie 4.2), tel qu'une meilleure expertise que l'IA sur les domaines abordés, nous pourrons alors utiliser correctement cet assistance qui n'aura de cesse d'innover dans le futur.

5 Conclusion

Dans ce rapport, nous avons abordé le sujet de l'IA générative de code en trois temps. Tout d'abord, nous avons évoqué brièvement les autres outils des programmeurs qui l'ont précédée par un historique et une mise en contexte de ceux-ci. Ensuite, nous nous sommes attardés plus longuement sur son fonctionnement en abordant les réseaux neuronaux et leur construction pour aboutir aux LLM et la méthode du fine tuning qui est la finalisation de l'entraînement de ce type de modèle. Pour terminer, nous avons étudié l'impact de l'IA générative de code sur l'industrie du développement, tout d'abord, en la confrontant aux outils évoqués en première partie, ce qui a résulté en un constat à la faveur de l'IA et de ses gains de productivité. Puis, nous avons décrit les apports du prompting face aux requêtes des moteurs de recherche traditionnels et évoqué les dangers et risques qu'entraîne son utilisation.

Pour compléter notre réponse à la problématique posée, l'IA générative de code est une avancée technologique en faveur de la productivité qui introduit de nouvelles manières d'assister le programmeur, cependant, elle doit être utilisée à bon escient car, notamment, comme nous l'avons vu (cf. partie 3.2, partie 4.3), de part sa nature probabiliste et sa base de données d'entraînement, elle peut induire des erreurs.

Dans le temps imparti, nous nous considérons satisfaits de la couverture de notre sujet au global. Nous avons pu détailler et développer avec suffisamment de profondeur la plupart des points importants afin de répondre à notre problématique. Néanmoins, avec un sujet aussi vaste que celui-ci et en plein cœur de l'actualité, il est certain que nous aurions pu aborder des notions supplémentaires ou en approfondir comme l'entraînement des réseaux de neurones artificiels et la descente du gradient qui est un algorithme essentiel dans celui-ci.

Il y a une question à laquelle nous aurions souhaité répondre mais que nous laissons ici en suspens. Aux vues de l'engouement autour de cette nouvelle technologie et des impacts majeurs qu'elle commence déjà à avoir à tous les niveaux de la société, qu'en est-il du point de vue législatif ? Nos élus se sont-ils déjà emparés de la question pour l'empêcher de faire des ravages entre de mauvaises mains, la loi ayant toujours du retard par rapport à l'innovation ? Quand n'est-il du droit d'auteur ?

Références

[1] E. Hilfrank, "Ada Lovelace, première programmeuse de l'Histoire – nationalgeographic.fr." https://www.nationalgeographic.fr/histoire/portrait-ada-lovelace-premiere-programmeuse-de-lhistoire.

[2] "Doxygen homepage — doxygen.nl." https://www.doxygen.nl/.

[3] "Qu'est-ce qu'un environnement de développement intégré (IDE) ? — redhat.com." https://www.redhat.com/fr/topics/middleware/what-is-ide.

[4] "Environnement de développement — Wikipédia — fr.wikipedia.org." https://fr.wikipedia.org/wiki/Environnement_de_d%C3%A9veloppement.

[5] Éditions Larousse, "intelligence artificielle - LAROUSSE — larousse.fr." https://www.larousse.fr/encyclopedie/divers/intelligence_artificielle/187257#:~:text=L%27intelligence%20artificielle%20.

[6] "Qu'est-ce que l'IA générative ? - Explication de l'intelligence artificielle générative - AWS — aws.amazon.com." https://aws.amazon.com/fr/what-is/generative-ai/.

[7] "Qu'est-ce qu'un agent conversationnel ? | IBM — ibm.com." https://www.ibm.com/fr-fr/topics/chatbots.

[8] "11.3 Neural network models | Forecasting : Principles and Practice (2nd.ed) — otexts.com." https://otexts.com/fpp2/nnetar.html.

[9] "Qu'est-ce qu'un réseau de neurones ? | IBM — ibm.com." https://www.ibm.com/fr-fr/topics/neural-networks.

[10] E. MARTIN, *Réseaux de neurones à l'échelle nano. Quels modèles d'apprentissage ?* PhD thesis, Université Paris-Saclay, 2022.

[11] S. d'Ascoli, *Comprendre la révolution de l'intelligence artificielle.* F1RST EDITIONS, 2020.

[12] "Qu'est-ce que l'apprentissage en profondeur ? | IBM — ibm.com." https://www.ibm.com/fr-fr/topics/deep-learning.

[13] "Apprentissage profond et apprentissage automatique - Azure Machine Learning — learn.microsoft.com." https://learn.microsoft.com/fr-fr/azure/machine-learning/concept-deep-learning-vs-machine-learning?view=azureml-api-2.

[14] "What is generative AI and how does it work ? – The Turing Lectures with Mirella Lapata — youtube.com." https://www.youtube.com/watch?v=_6R7Ym6Vy_I, 2023.

[15] "What are Transformers ? - Transformers in Artificial Intelligence Explained - AWS — aws.amazon.com." https://aws.amazon.com/what-is/transformers-in-artificial-intelligence/#:~:text=Transformers%20are%20a%20type%20of.

[16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, pp. 2–7, 2017.

[17] E. ZDNet, "Qu'est-ce que Stack Overflow ? Un forum pour celles et ceux qui codent - ZDNET — zdnet.fr." https://www.zdnet.fr/pratique/qu-est-ce-que-stack-overflow-un-forum-pour-celles-et-ceux-qui-codent-39945744.htm.

[18] "Fine-tuning (deep learning) - Wikipedia — en.wikipedia.org." https://en.wikipedia.org/wiki/Fine-tuning_(deep_learning).

[19] "GitHub Copilot - Wikipedia — en.wikipedia.org." https://en.wikipedia.org/wiki/GitHub_Copilot.

[20] "ChatGPT — Wikipédia — fr.wikipedia.org." https://fr.wikipedia.org/wiki/ChatGPT.

[21] "Les gains de productivité promis par l'intelligence artificielle mis en doute — lesechos.fr." https://www.lesechos.fr/monde/enjeux-internationaux/les-gains-de-productivite-promis-par-lintelligence-artificielle-mis-en-doute-2087857#:~:text=Les%20Ã¯conomistes%20de%20Goldman%20Sachs,prochaines%20annÃ©es%20aux%20Etats-Unis.

[22] L. rédaction d'ORSYS, "IA : les générateurs de code, une révolution pour les développeurs - ORSYS Le mag — orsys-lemag.com." https://orsys-lemag.com/ia-les-generateurs-de-code-une-revolution-pour-les-developpeurs/.

[23] "L'IA générative, palliatif aux manques de compétences IT - Le Monde Informatique — lemondeinformatique.fr." https://www.lemondeinformatique.fr/actualites/lire-l-ia-generative-palliatif-aux-manques-de-competences-it-92325.html.

[24] "AI-powered code completions with GitHub Copilot — code.visualstudio.com." https://code.visualstudio.com/docs/copilot/ai-powered-suggestions.

[25] "nira.com." https://nira.com/wp-content/uploads/2021/11/image1-3.png.

[26] "GitHub Copilot : Synthèse de l'IA pour les développeurs — geeek.org." https://www.geeek.org/github-copilot-astuces/.

[27] R. Kassel, "Prompt : Qu'est-ce que c'est ? Comment les utiliser ? — datascientest.com." https://datascientest.com/prompt-tout-savoir.

[28] https ://www.facebook.com/parentolivier/, "Maximiser l'efficacité de l'IA grâce à l'utilisation des prompts — olivierparent.com." https://www.olivierparent.com/ia_prompt/.

[29] "Functional specifications for AI software tools for electric power applications. Final report (Technical Report) | OSTI.GOV — osti.gov." https://www.osti.gov/biblio/5463284.

[30] "Bruno Legeard, Smartesting : "le test connaît aujourd'hui sa révolution industrielle" | LeMagIT — lemagit.fr." https://www.lemagit.fr/reponse/Bruno-Legeard-Smartesting-le-test-connait-aujourdhui-sa-revolution-industrielle.

[31] "OpenAI ajoute des fonctions de sécurité, de contrôle pour les entreprises - Le Monde Informatique — lemondeinformatique.fr." https://www.lemondeinformatique.fr/actualites/lire-openai-ajoute-des-fonctions-de-securite-de-controle-pour-les-entreprises-93610.html.

[32] S. Biget, "Les IA qui génèrent du code ajoutent des failles de sécurité ! — futura-sciences.com." https://www.futura-sciences.com/tech/actualites/intelligence-artificielle-ia-generent-code-ajoutent-failles-securite-102546/.

[33] V. R., "L'IA pourrait-elle entraîner l'abrutissement humain ? — intelligence-artificielle.com." https://intelligence-artificielle.com/ia-entrainer-abrutissement-humain/.